

Assignment #2:

Deploying a Containerized Machine Learning Pipeline on Kubernetes

Total score: 100

This assignment must be completed *individually*.

Due date: See the course page

Overview

In this assignment, you will build a machine learning pipeline, build and containerize the workload, deploy it on Kubernetes, configure GPU access, and benchmark the performance of a GPU-accelerated workload against CPU-only execution. You will create **a single Docker image and use it** for **both** the GPU verification task and the machine learning pipeline.

You will first verify GPU access from within a container running on a Kubernetes cluster. You will then build a machine learning pipeline for binary classification using the HIGGs dataset, train a Multilayer Perceptron (MLP), and compare its execution performance on CPU and GPU.

A **tutorial page** for creating and deploying a Docker container on a Kubernetes cluster is available at:

https://assessment.ecs.fullerton.edu/static/nautilus_tutorial.html/

Note that NRP Nautilus regularly updates its policies and procedures for accessing computing resources, so some steps in this tutorial may be outdated. In that case, you need to refer to the **NRP tutorial page**:

<https://nrp.ai/documentation/userdocs/tutorial/docker/>

Learning Objectives

By completing this assignment, you will be able to:

- Build Docker containers for computing workloads.
- Build a machine learning pipeline for binary classification.
- Deploy a containerized computational workload as a Kubernetes Job.
- Benchmark machine learning workloads on CPU and GPU.
- Analyze classification accuracy and CPU vs GPU performance.
- Identify performance tradeoffs and potential bottlenecks in a CPU/GPU execution pipeline.

Required Tasks

1. GPU verification (30 points)

(a) Build the Docker container.

Create one Docker container that provides the software environment for the entire assignment. The image must be capable of running the provided program “[container_nautilus_gpu_test.py](#)” for GPU verification.

Your container must:

- Install Python.
- Install all required dependencies.
- Copy the program file
- Execute the program automatically when the container starts.

You must provide a Dockerfile and a requirements.txt file.

Do not create a separate Docker image for Task 2.

(b) Deploy the container as a GPU-enabled Kubernetes Job.

Deploy your container to the Nautilus Hypercluster using Kubernetes.

Create a Kubernetes YAML file named “[deployment.yaml](#)”. Your YAML file must:

- Create exactly one Kubernetes Job. Do not create a standalone Pod.
- Request one GPU resource.
- Uses the Docker image created in Task 1 (a).
- Runs the program once and then exits successfully.
- Allow the program’s output to be viewed using “`kubectl logs`”.

This Kubernetes Job is used to verify that the Docker image can successfully access a GPU on the Nautilus Hypercluster.

The program must print the following information to standard output:

- Whether a GPU is available.
- The detected GPU device name
- Runtime timing information

Use “`kubectl logs`” to verify that the program successfully accessed the requested GPU. Include a screenshot of the Kubernetes logs in your report.

2. Machine learning pipeline for binary classification (70 points)

For this task, use the same Docker image created in Task 1. Do not create a second image. The image must contain the Python/PyTorch environment and dependencies required to run the machine learning pipeline.

Data set: HIGGS (<https://archive.ics.uci.edu/dataset/280/higgs>)

For this task, you will use the HIGGS dataset, which contains (Monte Carlo) simulated high-energy physics data for a binary classification problem involving the identification of a Higgs boson signal vers background events.

The dataset contains:

- 11,000,000 example
- 29 columns total. 1 class label (the first column), 28 features (21 low-level features and 7 high-level features derived from the low-level features)

For this assignment, use the data as follows:

- First 10,000,000 examples for training
- Last 1,000,000 examples for testing

(a) Design and configure the MLP (**20 points**)

Download the HIGGS dataset and implement a MLP for binary classification. Your MLP must have four layers, including the output layer, with the following architecture:

<i>Layer</i>	<i>Type</i>	<i>Number of neurons</i>	<i>Activation</i>
Input	Input features	28	
Hidden layer 1	Linear	300	ReLU
Hidden layer 2	Linear	300	ReLU
Hidden layer 3	Linear	100	ReLU
Output layer	Linear	1	None (raw logit value)

The output layer must produce a raw logit value. Since the output of the model will be a raw logit value, so, you need to convert it into a probability value (0.0-1.10) for binary classification, e.g., `torch.sigmoid(predicted_logit_value) > 0.5`.

Use the following training hyperparameters:

- Gradient optimizer: Adam
- Learning rate: 0.001
- Number of epochs: 5
- Loss function: Binary cross-entropy with logits (BCEWithLogitsLoss)

Answer the following questions:

- How many trainable parameters does the neural network contain, including biases?
- How many weights does the network contain, excluding biases?
- How many neurons are in the network, excluding the input features?

Show your calculation.

(b) Train and test the model (**20 points**)

Train the MLP using the 10,000,000 examples training set and evaluate it using the 1,000,000 example test set.

Report:

- Training results.
- Test accuracy as a percentage
- Any other relevant performance information you consider useful.

(c) Explain the deployment pipeline (**10 points**)

Briefly describe the deployment pipeline you used. Your explanation should include:

- How you built the Docker container.
- How the container was deployed to Kubernetes.
- Any challenges you encountered and how you resolved them.

Include relevant Kubernetes commands and configuration details where appropriate.

(d) Benchmark CPU vs GPU execution (**20 points**)

Measure and report the execution time of the pipeline using:

- CPU-only PyTorch execution
- GPU-accelerated PyTorch execution

For both experiments, use the same model architecture, dataset splits, batch size, optimizer, learning rate, number of epochs and other relevant configuration parameters.

Measure the execution time for the machine learning pipeline, including:

- Data loading
- Model initialization
- Model training
- Model testing/evaluation

Exclude the following from the benchmark:

- Downloading the dataset
- Building the Docker container image
- Building or pulling the container image

Report the execution time for both CPU and GPU execution and calculate the GPU speedup, if applicable, then briefly explain:

- Whether GPU execution was faster than CPU-only execution
- The magnitude of the performance difference
- Why GPU execution did or did not provide a significant speedup
- Any performance bottlenecks you observed
- How data loading, CPU-to-GPU data transfer, batch size, or other factors may have affected performance

Academic integrity and code-sharing policy

You may use publicly available source code or AI generated code as a reference. However, you must not share your code with other teams or students in the class.

Any violation of this policy will result in a zero score for this assignment and may result in additional academic consequences, including penalties on future assignments, in accordance with course and university policies.

Deliverables

Submit one report file in PDF or Word format containing:

Answers to all questions

- Answers to all questions
- MLP architecture and parameter calculations.
- Training and testing results.
- CPU-versus-GPU benchmark results.
- Description of the Kubernetes deployment pipeline.
- Explanation of GPU allocation and verification.
- Discussion of performance differences and bottlenecks.
- Discussion of any challenges encountered and how they were addressed.
- Screenshot(s) showing the Kubernetes Job logs.

Source code and configuration files:

- Docker files
- Requirements.txt
- YAML files
- Python program files

Grading Criteria

The assignment will be graded based on:

- Quality and clarity of the report and answers provided in the report
- Correctness of deployment and results
- Correctness of the machine learning implementation results
- Correctness and quality of the CPU/GPU benchmark
- Demonstrated understanding of containers, Kubernetes, GPU allocation, and machine learning pipelines
- Quality of performance analysis and discussion
- Effort and completeness (10% overall)